

Follow this 😊

It's okay. Take your time. No Rush.

Pick any problem.

1. Jot down your thoughts. How would you approach it?
What are the ideas you get.

2. If you are able to solve it, great! If not, try seeing the video. Does it give you hint of which direction to think in?

3. Using that, are you able to arrive at the solution.
If not, where are you getting stuck?

Mantras :-

① Participate in discussions

② Be active

③ Try thinking what is being asked

Recursion :-

- How to write Recursive codes } 1 hr
- Working } 1 hr
- Tc/Sc of recursive code } Next session

Why?

- Merge Sort / Quick Sort
- Binary Trees / BST / BBST
- Segment Trees / Tries
- Dynamic Programming
- Backtracking
- Graphs

Recursion :- function calling itself

↳ Solving a problem with smaller version of same problem
Subproblems

$$\text{Sum}(N) = 1 + 2 + 3 + 4 + \dots + N-1 + N$$

$\text{Sum}(N) = \text{Sum}(N-1) + N$

$\rightarrow \begin{aligned} \text{Sum}(5) &= 1 + 2 + 3 + 4 + 5 \\ \text{Sum}(5) &= \text{Sum}(4) + 5 \end{aligned}$

↓
Problem

↓
Smaller version of
same problem

↓
Subproblem

Recursive Code :-

- ① Assumption :- Decide what your funⁿ should do & assume it does
- ② Main logic :- Solve original problem with subproblem
- ③ Base Condition :- When should code stop.

```
int Sum(N) {
```

Ass:- Given N, calculate Sum of [1 N] & return it

```
if (N == 1) return 1
```

```
return Sum(N-1) + N
```

```
}
```

N=1

Sum(1) = Sum(0) + 1

Sum(1) = 1

$$3! = 3 \times 2 \times 1 \quad 4! = 4 \times 3 \times 2 \times 1 = 24.$$

$$N! = N \times (N-1) \times (N-2) \times \dots \times 1$$

→ Natural No $(N-1)! = \text{fact}(N-1)$

```
int fact(N) {
```

Ass: Given N, calculate N! & return it.

fact(3) = 6, fact(4) = 24

N=1 : fact(0) * 1

```
if (N == 1) return 1
```

```
return fact(N-1) * N
```

```
}
```

Fibonacci Series:-

						$N-2^{th}$	$N-1^{th}$	N^{th}	
1	2	3	4	5	6	7	8	9	10
1	1	2	3	5	8	13	21	34	55
						$\swarrow$	$\downarrow$		
						$fib(N-2)$	$fib(N-1)$		

```
int fib(N) {
```

Ass: Given N, calculate & return N^{th} fibonacci no.

```
if (N == 1 || N == 2) return 1
```

```
return fib(N-1) + fib(N-2);
```

```
}
```

$$\left\{ \begin{array}{l} N=3 \\ fib(3) = \check{fib(2)} + \check{fib(1)} \\ \boxed{fib(2)} = \check{fib(1)} + \underset{\times}{fib(0)} \\ \boxed{fib(1)} = \underset{\times}{fib(0)} + \underset{\times}{fib(-1)} \end{array} \right\}$$

Example:-

```
int add(N, M) {
    return N+M
}

int square(N) {
    return N*N
}

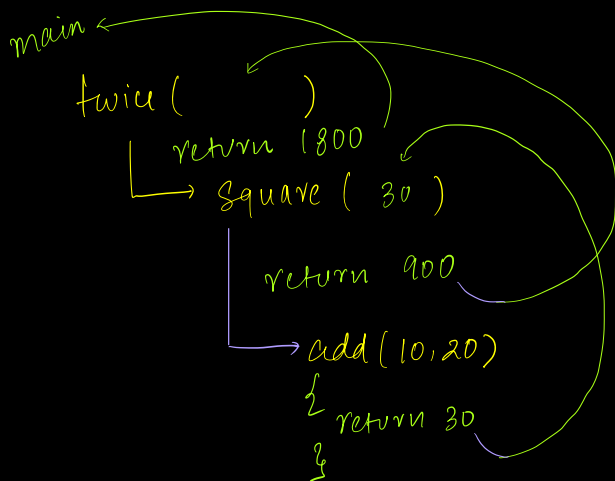
int twice(N) {
    return 2*N
}
```

main() {

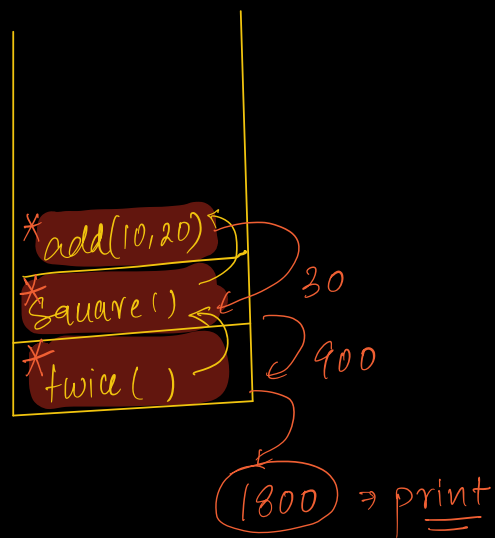
```
int x = add(10, 20) // 30
int y = square(x) // 900
int z = twice(y) // 1800
print(z) // 1800
}
```

main() {

```
print(twice(square(add(10, 20))))
// 1800
}
```



// Stack holds the function calls.



main() { sum(u) }

```

int Sum(N) {
    if (N==1) return 1
    return Sum(N-1) + N
}

```

$N = 4$
 $Sum(4)$
 $return \overset{6}{Sum(3)} + 4$

$Sum(N=3)$
 $return \overset{3}{Sum(2)} + 3$

$Sum(N=2)$
 $return \overset{1}{Sum(1)} + 2$

$Sum(N=1)$
 $return 1$

✓	$Sum(1) : return 1$
✓	$Sum(2) : return \overset{1}{Sum(1)} + 2$
✓	$Sum(3) : return \overset{3+3}{Sum(2)} + 3$
✓	$Sum(4) : return \overset{6+4}{Sum(3)} + 4$

$return \underline{10}$
 10

Given N , print all nos from 1 to N

$N=4$: 1 2 3 4

{ 1 2 3 4 ... $N-1$ } N }

void printEnc(N) {

{ Ass:- Given N , print all numbers from [1 2 3 ... N]

printEnc(5) :- 1 2 3 4 5

printEnc(4) :- 1 2 3 4

if ($N == 1$) { print(1); return; } → to come out of fun call.

printEnc($N-1$) // print all [1 $N-1$]

print(N) // print N th

} Main logic

}

main() { printEnc(4) }

printEnc(^N4) {

→ printEnc(3)

→ print(4)

}

printEnc(N=3) {

→ printEnc(2)

→ print(3)

}

printEnc(2) {

→ printEnc(1)

→ print(2)

}

printEnc(1) {

→ print(1)

→ return

}

void printEnc(N) {

if(N == 1) { print(1); return; }

printEnc(N-1)

print(N)

}

O/P

1 2 3 4

Given N , print all nos from $N, N-1, N-2, \dots, 1$

$N=5$ 5 4 3 2 1

TODO

Try it out H/W

Hint :- very very very minor change
in last code.

Break:-

Given a String, check Palindrome or not.

Ex $a \overset{x}{\underset{p1}{\uparrow}} c \ e \ \overset{x}{\underset{p2}{\uparrow}} g \ a \} \rightarrow \underline{\text{false}}$

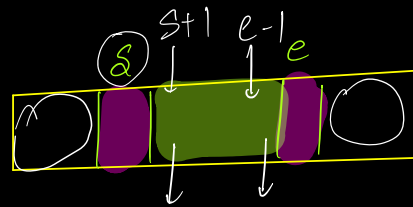
```
main() { str, 0, N-1
```

bool Palin (string str, s, e) {

Ass:- Given str, it will check whether substring [s e]
is palindrome or not.

if (s >= e) return true

str:-



if (str[s] == str[e]) {

palin(s+1, e-1)

if (palin(str, s+1, e-1)) {

return true

}

else {

return false

}

}

else {

return false

}

}

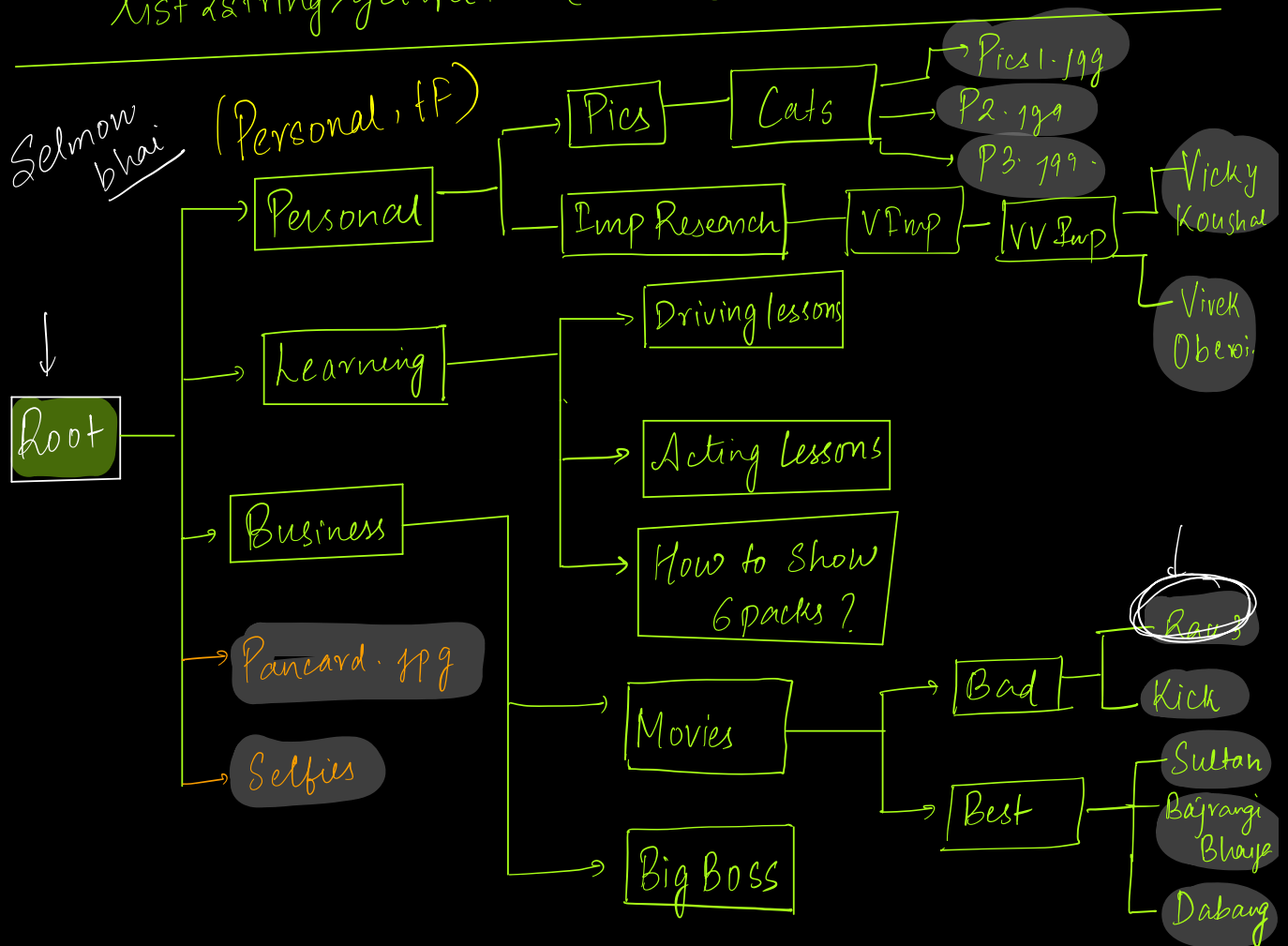
Google:-

Given a directory structure, search for a file it is there or not.

Inbuilt functions :-

`list <string> get All Directories (Directory Name)`

`list <string> get All Files (Directory Name)`



`get All Files (root) :-`

- Pancard
- Selfies

`get All Directories (learning) :-`

- Driving lesson
- Acting "
- 6 packs

```
bool Search(Directory Name, target File) {
```

```
    // Ass:- Check if target file is present in given directory.
```

```
    list<string> files = GetAllFiles(Dir Name)
```

```
    for(int i=0; i < files.size(); i++) {
```

```
        if (files[i] == target File) {
```

```
            return true
```

```
        }
```

```
    }
```

```
    list<string> dir = GetAllDirectories(Dir Name)
```

```
    for(int i=0; i < dir.size(); i++) {
```

```
        // check if target file is present dir[i]
```

```
        if (Search(dir[i], target File)) {
```

```
            return true
```

```
        }
```

```
    }
```

```
    return false
```

```
}
```